Практическая работа

«Построение графовых моделей объектов проектирования и строительства для последующего применения методов машинного обучения»

Цель работы: освоить методологию представления данных из области проектирования и строительства в виде графов различных типов и подготовить граф к решению задач машинного обучения (классификация узлов, предсказание рёбер, регрессия графа).

Задачи:

1. Научиться выделять сущности предметной области и переводить их в множества вершин и рёбер.

2. Освоить построение графов разных типов: ориентированных, взвешенных, атрибутированных, гетерогенных, динамических.

3. Сформировать матрицы признаков вершин и рёбер, пригодные для подачи в графовые нейронные сети (GCN, GraphSAGE, GAT).

4. Реализовать граф программно (NetworkX → PyTorch Geometric) и проверить его «машинную пригодность».

Инструменты: Python 3.10+, `networkx`, `pandas`, `numpy`, `scikit-learn`, `matplotlib`, `torch`, `torch_geometric` (или DGL).

Краткие теоретические сведения

Граф задаётся как G = (V, E), где V — множество вершин, E ⊆ V×V — множество рёбер. Для машинного обучения граф дополнительно снабжают:

- матрицей признаков вершин X ∈ R^(|V|×d);

- матрицей смежности A или списком рёбер edge_index размера 2×|E|;

- (опционально) признаками рёбер E_attr и метками y.

Типовые задачи МО на графах и их строительные примеры:

Задача	Суть	Пример в строительстве
Классификация вершин	предсказать метку узла	назначение помещения; уровень риска работы
Предсказание ребра	предсказать наличие/появление связи	недостающая технологическая зависимость; связь поставщик–объект
Классификация/регрессия графа	предсказать свойство всего графа	итоговая длительность проекта; класс энергоэффективности здания
Выделение сообществ	найти кластеры	группы взаимосвязанных помещений; бригады и фронты работ

Методические указания по составлению графов

3.1. Универсальный алгоритм (7 шагов)

Для любой предметной области выполните последовательно:

1. Определите вершины. Ответьте: «Что является неделимым объектом анализа?» (помещение, работа, поставщик, труба, норма).

2. Определите рёбра. Ответьте: «Какое отношение между сущностями несёт смысл?» (смежность, зависимость, поставка, регламентация).

3. Задайте направленность. Если отношение несимметрично («работа А предшествует В») — граф ориентированный (DiGraph). Если симметрично («помещения соединены дверью») — неориентированный.

4. Задайте веса. Вес ребра — измеримая величина отношения: длительность, стоимость, пропускная способность, расстояние.

5. Опишите признаки. Минимум 4 числовых и 1 категориальный признак на вершину; по возможности — признаки рёбер.

6. Выберите формат хранения: список рёбер (edge list, CSV), матрица смежности, таблицы признаков.

Проверьте качество графа (чек-лист в п. 3.3).

3.2. Типы графов в строительстве: шаблоны проектирования

Тип А. Пространственный граф здания (по данным BIM/IFC)

- Вершины: помещения / зоны (`IfcSpace`).

- Рёбра: наличие дверного проёма или общей границы (`IfcDoor`, `IfcRelSpaceBoundary`).

- Направленность: неориентированный (дверь двусторонняя).

- Признаки вершин: площадь, этаж, число окон, число дверей, высота потолка, тип отделки; метка — функциональное назначение (кабинет, коридор, санузел, серверная, склад).

- Задача МО: классификация назначения помещения по геометрии и связности; предсказание недостающих связей.

- Источник данных: экспорт из Revit/Archicad в IFC, парсинг через `ifcopenshell`.

Тип Б. Сетевой график строительства (календарный граф)

- Вершины: работы (задача календарного плана).

- Рёбра: технологические зависимости типов FS, SS, FF, SF.

- Направленность: строго ориентированный (ациклический — проверить на отсутствие циклов!).

- Вес ребра: лаг (смещение) между работами; признаки вершин: длительность, стоимость, число рабочих, тип бригады.

- Задача МО: классификация работ по риску задержки; регрессия длительности всего проекта; предсказание отсутствующих зависимостей.

- Источник данных: импорт из MS Project (XML) или Primavera (XER): поля «Задачи» и «Связи».

Тип В. Гетерогенный граф логистики снабжения

- Вершины нескольких типов: поставщик, материал, склад, объект, техника.

- Рёбра нескольких типов: «поставляет», «хранится», «перевозится», «закреплена за».

- Задача МО: предсказание связи «поставщик–объект» (рекомендация), классификация узких мест.

- Особенность: для PyG такой граф оформляется через `HeteroData` с разными `edge_index` для каждого типа отношений.

Тип Г. Граф инженерных сетей микрорайона

- Вершины: узлы сети (колодцы, задвижки, ТП, потребители).

- Рёбра: участки трубопровода / кабеля; вес — диаметр, длина, пропускная способность.

- Тип: взвешенный, допускаются мультирёбра → использовать `MultiGraph`.

- Задача МО: классификация аварийности участков, выделение зон обслуживания.

Тип Д. Динамический граф производства работ

- Снапшоты графа на еженедельные даты: вершины — фронты работ, рёбра — активные взаимодействия бригад.

- Задача МО: прогноз перехода объекта в состояние «срыв срока» с помощью временных графовых моделей (например, эволюция признаков).

3.3. Чек-лист «пригоден ли граф для машинного обучения»

Перед переходом к модели убедитесь:

- число вершин ≥ 100 (при меньшем — объедините графы всей группы в один датасет);

- каждая вершина имеет вектор признаков одинаковой длины;

- числовые признаки нормализованы (StandardScaler/MinMax), категориальные — закодированы (one-hot или label-encoding);

- для ориентированных графов при использовании неориентированных сверток добавлены обратные рёбра (`to_undirected`);

- нет изолированных вершин без признаков (либо они явно помечены);

- метки известны хотя бы для части вершин (обучение с учителем — сплит 60/20/20);

- оценён дисбаланс классов; при сильном дисбалансе запланированы веса классов или стратификация.

4. Практическое задание

Часть 1. Построение графа по варианту

Для своей предметной области:

1. Сформулируйте множества V и E в виде таблиц.

2. Составьте `nodes.csv` (id + ≥4 числовых признака + 1 категориальный + метка) и `edges.csv` (src, dst, weight, при необходимости — тип ребра).

3. Постройте граф в NetworkX и визуализируйте.

4. Рассчитайте и запишите метрики: |V|, |E|, плотность, средний коэффициент кластеризации, распределение степеней, топ-5 вершин по центральности; для орграфа — проверку ацикличности.

Варианты:

1. Пространственный граф этажа административного здания (Тип А).

2. Сетевой график возведения каркаса 5-этажного дома (Тип Б).

3. Гетерогенный граф снабжения стройплощадки (Тип В).

4. Граф теплоснабжения микрорайона (Тип Г).

5. Граф знаний: нормы (СП/ГОСТ) ↔ элементы здания (Тип Д, упрощённый).

6. Динамический граф взаимодействия субподрядчиков по 4 этапам проекта.

Пример исходных данных для Варианта 1 (фрагмент)

Таблица вершин `nodes.csv`:

id	name	area	floor	windows	doors	height	label
101	Кабинет 1	24.5	1	2	1	3.0	office
102	Кабинет 2	18.0	1	2	1	3.0	office
103	Коридор	32.0	1	0	8	3.0	corridor
104	Санузел	6.2	1	1	1	3.0	wc
105	Серверная	12.0	1	0	1	3.0	server
106	Холл	40.0	1	3	4	3.2	lobby

Таблица рёбер `edges.csv` (дверь между помещениями):

src	dst	weight
101	103	1
102	103	1
104	103	1
105	103	1
106	103	1

Студент обязан расширить фрагмент минимум до 25 помещений и 30 связей.

Часть 2. Кодирование признаков для графовых нейросетей

1. Нормализуйте числовые признаки, закодируйте категориальные.

2. Сформируйте тензоры `x`, `edge_index`, `y` и объект `torch_geometric.data.Data`.

3. Обоснуйте выбор признаков: какие из них, по вашему мнению, информативны для целевой задачи.

Шаблон кода:

```
import pandas as pd, torch
from sklearn.preprocessing import StandardScaler, LabelEncoder
from torch_geometric.data import Data
```

```
nodes = pd.read_csv("nodes.csv")
edges = pd.read_csv("edges.csv")

num_cols  = ["area", "floor", "windows", "doors", "height"]
nodes[num_cols] = StandardScaler().fit_transform(nodes[num_cols])

le = LabelEncoder()
nodes["label"] = le.fit_transform(nodes["label"])

x = torch.tensor(nodes[num_cols].values, dtype=torch.float)
y = torch.tensor(nodes["label"].values, dtype=torch.long)

edge_index = torch.tensor(edges[["src", "dst"]].values.T, dtype=torch.long)
# для неориентированного представления добавляем обратные рёбра
edge_index = torch.cat([edge_index, edge_index.flip(0)], dim=1)

data = Data(x=x, edge_index=edge_index, y=y)
print(data)
```

Часть 3. Базовая модель машинного обучения на графе

В зависимости от варианта выполните одну из задач:

- Классификация вершин. Разметьте метки для части вершин (маски `train/val/test`), обучите GCN или GraphSAGE из 2 слоёв, приведите `accuracy` и `confusion_matrix`.

- Предсказание рёбер. Скройте 15–20% рёбер, обучите модель на оставшихся, восстановите скрытые связи и оцените AUC-ROC.

- Регрессия графа. Если граф небольшой — сформулируйте задачу как предсказание глобального показателя (например, суммарной длительности) и используйте `global_mean_pool`.

Часть 4. Отчёт

Отчёт (PDF, 6–10 стр.) должен содержать:

1. Описание предметной области и обоснование выбора типа графа.

2. Таблицы `nodes.csv`, `edges.csv` (или скриншоты).

3. Визуализацию графа и рассчитанные метрики.

4. Код построения `Data`-объекта.

5. Описание модели, метрики качества, анализ ошибок.

6. Выводы: какие строительные решения можно принять на основе полученной модели (например, «модель позволяет автоматически классифицировать помещения при импорте IFC и находить ошибки проектирования»).

Контрольные вопросы

1. Чем графовое представление отличается от табличного? Какую информацию теряет таблица на примере здания?

2. Матрица смежности и список рёбер: достоинства, недостатки, когда что применять?

3. Какие типы графов применяются в строительстве? Приведите по одному примеру.

4. Почему сетевой график строительства обязан быть ациклическим? Как проверить это программно?

5. Как кодируются категориальные признаки вершин для подачи в GNN?

6. Чем отличается гетерогенный граф от однородного? Пример из логистики стройплощадки.

7. Как из модели информационного здания (IFC) извлечь пространственный граф?

8. Что такое задача предсказания рёбер и как она применяется при проверке проектных зависимостей?

9. Зачем нормализовать признаки вершин перед обучением графовой нейросети?

10. Как бороться с дисбалансом классов вершин (например, подсобных помещений меньше, чем кабинетов)?

Рекомендуемые источники

1. Документация PyTorch Geometric: Getting Started with GNNs.

2. NetworkX documentation — Graph generators and algorithms.

3. buildingSMART. IFC 4.3 — классы `IfcSpace`, `IfcDoor`, `IfcRelSpaceBoundary`.

4. Методические материалы курса по сетевому планированию (метод критического пути).

5. Стоянов Е. А. Графовые нейронные сети в задачах анализа строительных данных (конспект лекций курса).

https://blog.deepschool.ru/math/zadachi-mashinnogo-obucheniya-na-grafah/

https://www.alphaxiv.org/ru/abs/2303.14483